

MedChain AI Executive Suite

Decentralized Organ Allocation Engine, DeepSurv Analytics & Blockchain Provenance Architecture

Document Type: Complete System User Guide & Architecture Report

Date: October 06, 2026

Core Domain: Healthcare AI, Survival Analytics, EVM Blockchain

Version: 1.0.0 (Production Executive Suite)

1. Executive Summary & Purpose

MedChain AI is an enterprise-grade, decentralized organ allocation platform designed to revolutionize donor-recipient organ matching, 5-year post-transplant survival prediction, and multi-hospital consortium organ distribution. Historically, organ allocation systems suffer from regional allocation sub-optimality, lack of transparency, and susceptibility to data tampering. MedChain addresses these challenges by combining state-of-the-art machine learning models with privacy-preserving bipartite graph optimization and immutable Smart Contract provenance.

Key Objectives & System Capabilities:

- **High-Precision Machine Learning Inference:** Predicts donor-recipient organ compatibility scores and 5-year post-transplant survival probabilities using Gradient Boosting classifiers.
- **Explainable AI & Survival Trajectories:** Computes SHAP (SHapley Additive exPlanations) feature attributions and models 60-month Kaplan-Meier hazard curves.
- **Phase 2 Global Bipartite Graph Optimization:** Uses the Hungarian Algorithm (Kuhn-Munkres algorithm) across a 5-hospital consortium network to maximize total organ compatibility while preserving 100% data privacy via score-only matrices.
- **Phase 3 Blockchain Audit & Provenance:** Commits every approved organ allocation tuple to an Ethereum Virtual Machine (EVM) Solidity smart contract (MedChainLedger.sol), producing Keccak256 digests and ERC-721 Virtual NFT Digital Twin certificates.

2. End-to-End Technology Stack & Architecture

The MedChain platform is architected as a high-performance multi-tier software system combining Python analytical microservices, a dynamic React SPA frontend, and an Ethereum EVM smart contract layer.

Component Layer	Technologies Used	Core Functional Role
Backend Engine	FastAPI, Uvicorn, Python 3.10+	RESTful API services, RPC proxying, Hardhat node auto-launcher.
AI / ML Models	Scikit-Learn, Lifelines, Scipy, Joblib, Pandas, NumPy	Gradient Boosting Classifier, Kaplan-Meier fitter, Hungarian bipartite optimizer.
Frontend UI	React 18, Vite, Tailwind CSS, Framer Motion, Recharts, Lucide	Modern glassmorphism interface, interactive visual graph canvas, real-time telemetry.

Blockchain & Provenance	Solidity ^0.8.20, Hardhat, Ethers.js v6, Keccak256, Hyperledger Fabric concept	Smart contract ledger (MedChainLedger.sol), ERC-721 NFT twin, on-chain minting.
Export & Utilities	html2canvas, jsPDF, react-qr-code, ReportLab	Generating downloadable PDF smart contract certificates & executive user guides.

3. Detailed Guide to All Tabs, Menus & Interface Modules

The MedChain Executive Dashboard is organized into a top persistent navigation header and five dedicated multi-tab viewports.

A. Global Header & Action Controls

- **FastAPI Health Connection Badge:** Displays live status of the backend engine (FastAPI Connected glowing green vs Offline Mode amber warning with reconnect button).
- **Preset Clinical Vectors:** One-click buttons to populate sample donor-recipient profiles: **Optimal** (92% survival baseline), **Marginal** (72% survival), and **High Risk** (55% survival).
- **Multi-Tab Navigation Bar:** Enables instant seamless switching between all 5 system modules with active route indicators.

B. Tab 1: Consortium Network Map (Hyperledger Fabric Mesh)

This tab visualizes the 5-hospital federated consortium network that synchronizes anonymized organ vectors across regions:

1. **HOS-01: Johns Hopkins Organ Center** (Baltimore, MD) — Node ping ~14ms
2. **HOS-02: Mayo Clinic Transplant Hub** (Rochester, MN) — Node ping ~18ms
3. **HOS-03: Mass General Allocation Node** (Boston, MA) — Node ping ~12ms
4. **HOS-04: Stanford Medical Blockchain** (Palo Alto, CA) — Node ping ~24ms
5. **HOS-05: Cleveland Clinic FedNet** (Cleveland, OH) — Node ping ~16ms

Key Features: Interactive SVG peer topology map with animated packet flows, peer telemetry inspector (CPU load, committed block height, PBFT consensus state, mTLS channel encryption, and chaincode medchain-cc:v3.2 status).

C. Tab 2: Clinical Match Engine (Vector Form Inputs)

This module allows clinical team members to input biological parameters for a donor-recipient pair:

- **Patient Age (18 - 85 yrs) & Donor Age (15 - 80 yrs):** Interactive sliders auto-computing the Absolute Age Differential (Δ).
- **Patient BMI (14 - 50 kg/m²) & Donor Weight (35 - 150 kg):** Precise biometric inputs.
- **RealTime Organ Health (0 - 100) & Organ Condition Rating (1 - 10):** Perfusion & viability parameters.
- **HLA / Organ Match Rating (Grade 1 to 5):** Immunological structural match level buttons.
- **Blood Group Compatibility Toggle:** ABO/Rh cross-match switch (Compatible vs Incompatible).
- **Baseline Survival Estimate:** Initial clinical prognosis input.

Action: Clicking 'Evaluate Organ Allocation & Survival' transmits the vector to the FastAPI endpoint /api/predict-match.

D. Tab 3: DeepSurv Analytics (AI Predictions & Explainability)

Presents comprehensive analytical outputs returned by the AI engine:

- **AI Recommendation Badge:** Displays APPROVED (Match score $\geq 70\%$) or REJECTED_BY_AI.
- **4-Stage AI Stepper:** Vector Normalization → Gradient Boosting → SHAP Attributions → Kaplan-Meier Curves.
- **KPI Cards:** AI Match Confidence Score (%), 5-Year Survival Expectancy (%), and DeepSurv Hazard Ratio (Relative Risk).
- **SHAP Feature Importances Chart:** Recharts horizontal bar chart depicting top 5 clinical feature weights driving the AI decision.
- **Kaplan-Meier Survival Probability Curve:** 60-month longitudinal line chart tracking patient survival trajectory.

E. Tab 4: Smart Contract Ledger & NFT Provenance

Translates approved AI decisions into permanent, tamper-proof blockchain records:

- **Hardhat EVM Status & Live Terminal:** Real-time STDOUT log window capturing consensus rounds and block proposals.
- **On-Chain Minting Engine:** Executes `MedChainLedger.mintMatchRecord(...)` to create a Keccak256 hash digest of the match tuple.
- **Official High-Grade Certificate:** Styled UI certificate with United States Organ Allocation Authority emblem, serial number, organ specs, Keccak256 digest, scannable QR code, signature lines, and official EVM seal.
- **Virtual 3D Holographic NFT Twin:** ERC-721 token card showing unique Token ID, IPFS metadata URI, and spinning organ core.
- **PDF Export:** Integrated canvas exporter allowing clinicians to download official PDF certificates.

F. Tab 5: Global Graph Matching (Phase 2 Hungarian Optimization)

Solves the multi-hospital organ distribution problem using linear assignment optimization:

- **100% Privacy Preservation:** Uses Keccak256/SHA-256 anonymized hashes and score-only matrices (Zero raw EHR exposure).
- **Hungarian Engine:** Runs `scipy.optimize.linear_sum_assignment` to maximize total compatibility.
- **Impact Metrics:** Displays Global Hungarian Score vs Greedy Baseline Score, Network Efficiency Gain (+%), and Life-Years Saved (+Years).
- **View Modes:** Global Hungarian Optimal Bipartite Web, Greedy Sequential Baseline, and 10x10 Anonymized Adjacency Heatmap Matrix.

4. End-to-End Operational Workflow

Follow this step-by-step procedure when operating the MedChain platform:

- **Step 1: System Initialization:** Launch FastAPI backend (`uvicorn main:app --reload`) which automatically verifies/starts the local Hardhat EVM node on port 8545 and deploys `MedChainLedger.sol`.
- **Step 2: Network Audit:** Navigate to Tab 1 ('Consortium Network') to verify that all 5 hospital peer nodes are online and synchronized with low latency.

- **Step 3: Clinical Parameter Entry:** Navigate to Tab 2 ('Clinical Match Engine'). Select a preset (e.g. 'Optimal') or adjust sliders for donor-recipient parameters. Click 'Evaluate Organ Allocation & Survival'.
- **Step 4: Review AI & Survival Analytics:** Tab 3 ('DeepSurv Analytics') automatically opens. Review the AI Match Confidence Score, 5-Year Survival Expectancy, SHAP feature importances, and Kaplan-Meier trajectory curve.
- **Step 5: Blockchain Provenance Minting:** Click 'Proceed to Blockchain' to switch to Tab 4 ('Smart Contract Ledger'). Click 'Mint Smart Contract to Hyperledger / EVM'. View the generated official certificate and download the PDF.
- **Step 6: Global Bipartite Optimization:** Navigate to Tab 5 ('Global Graph Matching'). Click 'Run Global Bipartite Optimization' to observe network-wide Hungarian optimization across all 10 donor-recipient pool pairs.

5. Smart Contract Specifications (MedChainLedger.sol)

Below is the core Solidity smart contract deployed on the EVM blockchain layer for storing immutable organ match records:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract MedChainLedger {
    struct MatchRecord {
        string patientId;
        string donorId;
        string organType;
        uint256 matchScore;
        uint256 timestamp;
        bytes32 recordHash;
        address registeredBy;
    }

    mapping(bytes32 => MatchRecord) public records;
    bytes32[] public recordHashes;
    event MatchMinted(bytes32 indexed recordHash, string patientId, string donorId, uint256 matchScore);

    function mintMatchRecord(string memory _patientId, string memory _donorId,
        string memory _organType, uint256 _matchScore) public returns (bytes32, uint256) {
        uint256 currentTimestamp = block.timestamp;
        bytes32 recordHash = keccak256(abi.encodePacked(_patientId, _donorId, _organType, _matchScore, currentTimestamp, msg.sender));
        records[recordHash] = MatchRecord(_patientId, _donorId, _organType, _matchScore, currentTimestamp, recordHash, msg.sender);
        recordHashes.push(recordHash);
        emit MatchMinted(recordHash, _patientId, _donorId, _matchScore);
        return (recordHash, currentTimestamp);
    }
}
```

6. Deployment & Environment Setup Guide

Backend Setup:

1. Install Python dependencies: `pip install -r requirements.txt`
2. Run FastAPI server: `uvicorn main:app --reload --port 8000`

Frontend Setup:

1. Change directory: `cd frontend`
2. Install packages: `npm install`
3. Start Vite dev server: `npm run dev`

Conclusion: MedChain AI presents a groundbreaking fusion of machine learning, privacy-preserving graph algorithms, and EVM blockchain technology, delivering an uncompromised solution for transparent and life-saving organ allocation.